

AI Cognitive Runtime — Enterprise Technical Manual

Versão: 1.0.0

Data: 2026-05-19

Projeto: ai-lab / ai-cognitive-runtime

Organização: VRASHOWS — HUB premium de soluções integradas para eventos corporativos

Classificação: Interno — Técnico

Índice

- [1. Visão Geral da Plataforma](#)
- [2. Arquitetura Completa](#)
- [3. Agentes Implementados](#)
- [4. Sistema de Memória e RAG](#)
- [5. Email Delivery System](#)
- [6. Attachment System](#)
- [7. Lead Enrichment System](#)
- [8. Outbound Strategy](#)
- [9. Cost Optimization System](#)
- [10. Configuração Completa](#)
- [11. Execução dos Agentes](#)
- [12. Git e Versionamento](#)
- [13. Troubleshooting Completo](#)
- [14. Deployment em Outras Empresas](#)
- [15. Roadmap Futuro](#)

1. Visão Geral da Plataforma

1.1 O que é o AI Cognitive Runtime

O **AI Cognitive Runtime** é uma plataforma de infraestrutura AI-nativa, construída em TypeScript, projetada para orquestrar agentes de linguagem autônomos com memória persistente, roteamento inteligente de modelos, rastreamento de custo e pipelines de multi-agentes. A plataforma é o motor operacional por trás das capacidades de outreach inteligente da VRASHOWS.

A plataforma foi concebida como um **cognitive operating system** — não um chatbot, não uma automação de scripts, mas uma infraestrutura orquestrável onde agentes especializados colaboram em tarefas complexas de negócio: pesquisa de prospects, enriquecimento de leads, geração de mensagens de outreach calibradas por segmento, e entrega rastreável de emails corporativos.

1.2 Contexto de Negócio — VRASHOWS

A VRASHOWS é um **HUB premium de soluções integradas para eventos corporativos e experiências de marca**.

O tagline que orienta toda a comunicação da plataforma:

"Enquanto você fecha negócios, nós controlamos a operação."

A plataforma AI foi construída para escalar a estratégia de outbound enterprise da VRASHOWS — identificar decisores em empresas-alvo, inferir dados de contato corporativos, gerar comunicações posicionadas como parceiro estratégico operacional (não fornecedor), e executar campanhas de email rastreáveis com media kit anexado.

O que a VRASHOWS entrega:

- Coordenação e logística de eventos corporativos enterprise
- Staff especializado e hospitalidade executiva
- Produção e experiência do cliente em stands e lounges
- Cobertura em tempo real e conteúdo ao vivo
- Controle operacional 360° em feiras, exposições e ativações de marca

Referência de caso: Operação 360° da Brasil TecPar na ABRINT 2026.

Quem contrata a VRASHOWS: Diretores e VP de Marketing, Eventos, Experiência do Cliente, Comunicação Corporativa, Patrocínio — os responsáveis pela presença de marca em feiras enterprise.

1.3 Princípios de Design

A plataforma foi construída sobre seis princípios fundamentais:

Princípio	Descrição
Tool-first execution	Agentes priorizam: cache → retrieval → database → tools → APIs → raciocínio. Reasoning é o último recurso.
Memory-aware	Agentes recuperam memória semântica antes de raciocinar. Evitam contexts gigantes. Comprimem histórico. Reutilizam conhecimento.
Cost optimization	Haiku para tarefas leves. Sonnet para orquestração/código. Opus para planejamento/reflexão. Sem chamadas desnecessárias.
Observability	Todos os agentes expõem: token usage, latência, custo, métricas de retrieval, tool usage, workflow tracing.
Modularity	Um agent por folder. Prompts versionados como markdown. Tools stateless e reutilizáveis.
Graceful degradation	Infraestrutura ausente (Redis, Postgres) não crasha — degradação elegante para no-ops.

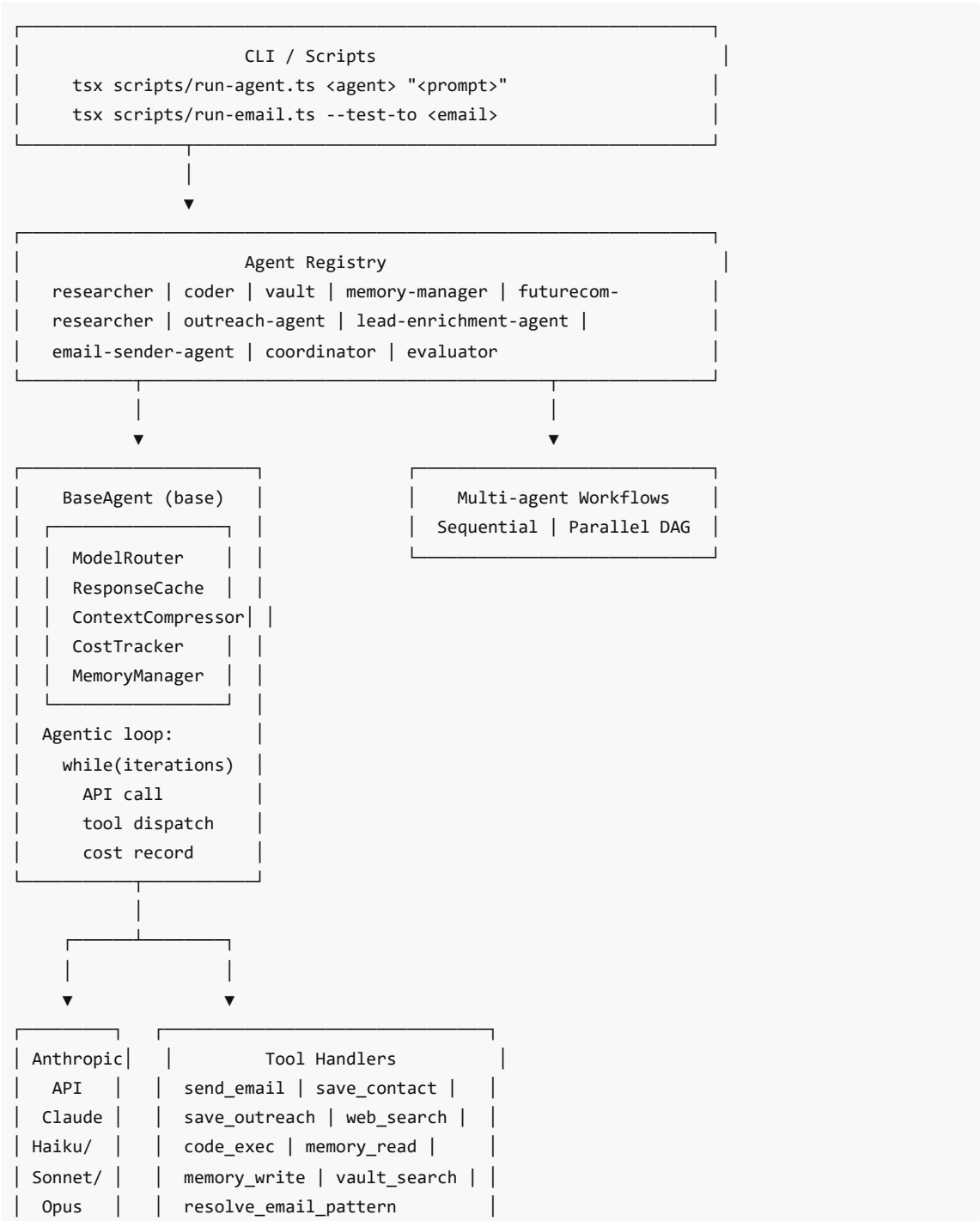
1.4 Stack Tecnológico

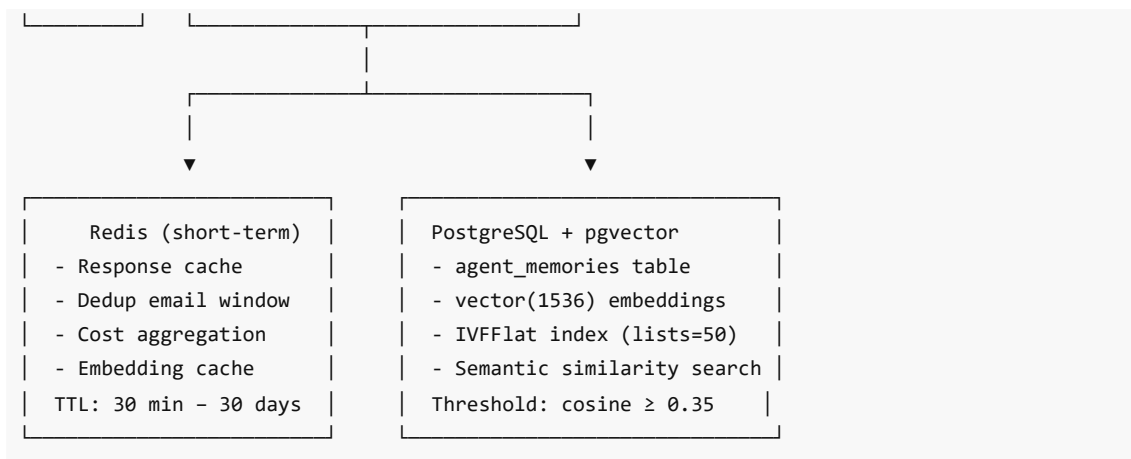
Componente	Tecnologia	Versão
Runtime	Node.js (ESM, "type": "module")	22.x
Linguagem	TypeScript	^5.7.0
Executor de dev	tsx (sem build step)	^4.19.0
LLM SDK	@anthropic-ai/sdk	^0.39.0
Email delivery	resend	^6.4.2
Memória curto prazo	Redis via redis	^4.7.0
Memória longo prazo	PostgreSQL + pgvector via pg	^8.13.0
Embeddings	OpenAI text-embedding-3-small	via openai ^6.38.0
RAG/Documents	LangChain	^0.3.0

Validação	Zod	^3.23.0
Logging	Winston	^3.17.0
Concorrência	p-limit, p-retry	^6.x

2. Arquitetura Completa

2.1 Visão de Alto Nível





2.2 Estrutura de Diretórios Detalhada

```

ai-cognitive-runtime/
├── agents/
│   ├── _base/                # Infraestrutura compartilhada
│   │   ├── agent.ts          # BaseAgent – agentic loop, tool dispatch, cost tracking
│   │   ├── cache.ts          # ResponseCache – Redis-backed response memoization
│   │   ├── context.ts        # estimateTokens() + compressContext()
│   │   ├── router.ts         # ModelRouter – heuristic + LLM-classifier routing
│   │   └── types.ts           # AgentConfig, AgentResult, ToolHandler, AgentStep
│   ├── researcher/           # Agente de pesquisa geral
│   ├── coder/                # Agente de geração de código
│   ├── vault/                # Agente de busca no Obsidian vault
│   ├── memory-manager/       # Extrai e consolida memórias de sessões
│   ├── evaluator/            # Loop de reflexão e critique
│   ├── coordinator/          # Orquestrador multi-agente
│   ├── futurecom-researcher/ # Pesquisa leads em feiras enterprise
│   │   ├── agent.ts
│   │   ├── schemas.ts        # Zod schema + JSON Schema para save_lead
│   │   └── types.ts           # LeadProfile interface
│   ├── outreach-agent/       # Gera pacotes de outreach personalizados
│   │   ├── agent.ts
│   │   ├── schemas.ts        # OutreachPackage validation
│   │   └── types.ts
│   ├── lead-enrichment-agent/ # Enriquece contatos com emails inferidos
│   │   ├── agent.ts
│   │   ├── email-resolver.ts  # EmailPatternResolver – 40+ company registry
│   │   ├── schemas.ts
│   │   └── types.ts           # EnrichedContact, GuessedEmail, EmailPatternResult
│   ├── email-sender-agent/   # Despacha emails via Resend
│   │   └── agent.ts
  
```

```

| | | └─ schemas.ts          # sendEmailSchema – Zod validation
| | |   └─ types.ts          # EmailRecord, SendEmailRequest
| | |
| | └─ registry.ts          # Mapa de nomes → factories (para scripts)
| |
| └─ tools/                 # Tool handlers reutilizáveis (stateless)
|   │ └─ index.ts           # Re-exports: memoryReadTool, memoryWriteTool
|   │ └─ send-email.ts      # sendEmail() + createSendEmailTool()
|   │ └─ web-search.ts      # Tavily web search tool
|   │ └─ code-exec.ts       # Execução segura de código (sandbox)
|   │ └─ vault-search.ts    # Busca semântica no Obsidian vault
|   │ └─ memory-tool.ts     # memory_read / memory_write tools
|   │
|   └─ memory/
|     │ └─ manager.ts       # MemoryManager – pgvector store + embeddings
|     │ └─ compressor.ts    # MemoryCompressor – sumarização de histórico
|     │   └─ short-term/
|     │     └─ redis.ts     # RedisMemory – get/set/del com degradação graceful
|     │
|     └─ config/
|       │ └─ env.ts         # Zod schema + safeParse de process.env
|       │ └─ models.ts      # Models const + getMaxTokens() + getMaxIterations()
|       │ └─ costs.ts       # MODEL_PRICING + calculateCost() + recordCost()
|       │ └─ routing.ts     # COMPLEXITY_SIGNALS + ROUTING_THRESHOLDS
|       │ └─ dynamic-routing.ts # Routing dinâmico avançado
|       │ └─ logger.ts      # Winston logger (info, warn, error, debug)
|       │
|       └─ workflows/      # Orquestração multi-agente
|       │
|       └─ prompts/
|         │ └─ agents/      # System prompts versionados como .md
|         │   │ └─ outreach-agent.md
|         │   │ └─ email-sender-agent.md
|         │   │ └─ lead-enrichment-agent.md
|         │   │ └─ futurecom-researcher.md
|         │   │
|         │   └─ assets/
|         │     │ └─ pdfs/
|         │     │   │ └─ vrashows_media_kit_optimized.pdf # Gitignored – não commitado
|         │     │   └─ templates/
|         │     │     │ └─ cold-outreach.md
|         │     │     │ └─ follow-up.md
|         │     │     │ └─ re-engagement.md
|         │     │     │ └─ executive-intro.md
|         │     │     └─ pipeline-schema.json
|         │     └─ leads/
|         │
|         └─ data/
|           │ └─ leads/
|           │   │ └─ aws-leads.json # 6 decisores AWS LATAM
|           │   └─ outreach/      # Gitignored – artefatos gerados
|           │

```

```

├─ logs/
│   └─ outreach/                # Gitignored – logs de entrega
│
├─ scripts/
│   ├── run-agent.ts            # CLI principal de agentes
│   ├── run-email.ts            # CLI de envio de email
│   ├── run-eval.ts             # Runner de evals
│   ├── cost-report.ts          # Relatório de custo por agente
│   ├── index-vault.ts          # Indexação do Obsidian vault
│   └─ memory.ts                # CLI de manutenção de memória
│
├─ infra/
│   └─ postgres/
│       └─ init.sql              # Inicialização automática com extensão vector
│
├─ evals/                       # Eval runner contra modelos live
├─ obsidian-vault/              # Vault local de memória arquitetural
├─ docs/
│   ├── ADR-001-architecture.md
│   └─ AI_RUNTIME_ENTERPRISE_MANUAL.md # Este documento
│
├─ docker-compose.yml           # Redis + PostgreSQL/pgvector
├─ package.json
├─ tsconfig.json
├─ .env                         # NUNCA commitar
├─ .env.example                 # Template público sem secrets
└─ .gitignore

```

2.3 Fluxo do Agentic Loop

O coração da plataforma é o método `BaseAgent.run()` em `agents/_base/agent.ts`. Abaixo o fluxo completo de execução para cada chamada:

```

run(userMessage, options)
|
├─ 1. MODEL ROUTING
|   ├── if model == "auto" → ModelRouter.route(prompt)
|   |   ├── scoreHeuristics(prompt) → score + signals
|   |   ├── lengthBonus(prompt) → +0/+5/+10/+20
|   |   ├── if confident (score far from thresholds) → heuristic decision
|   |   └─ if ambiguous → classifyWithLLM(prompt) via Haiku (max_tokens=10)
|   └─ Cheap mode override: always → Models.fast (Haiku)
|
├─ 2. RESPONSE CACHE CHECK
|   └─ if enableResponseCache && no tools in flight:
|       └─ Redis lookup by hash(model + systemPrompt + userMessage)
|           └─ HIT → return immediately (no API call)
|
├─ 3. MEMORY INJECTION
|   └─ if memoryEnabled:
|       └─ MemoryManager.getContextFor(agentName, userMessage)
|           └─ Semantic search → top-5 relevant memories → append to systemPrompt

```

```

└─ 4. AGENTIC LOOP (while iterations < maxIterations)
    └─ a. CONTEXT COMPRESSION CHECK
        └─ estimateTokens(messages) > contextTokenLimit (80k default)
            └─ compressContext() → summarize older turns → replace with summary

    └─ b. ANTHROPIC API CALL
        └─ client.messages.create({
            model: resolvedModel,
            max_tokens: getMaxTokens(),
            system: [{ type:"text", text: effectiveSystemPrompt,
                cache_control: { type: "ephemeral" } }],
            tools: [...toolSchemas],
            messages: [...conversationHistory]
        })

    └─ c. USAGE ACCUMULATION
        └─ inputTokens + outputTokens + cacheReadTokens + cacheCreationTokens

    └─ d. RESPONSE DISPATCH
        └─ stop_reason == "end_turn" → set finalOutput → break
        └─ stop_reason == "tool_use" →
            └─ emit onStep("tool_call")
            └─ handler = toolHandlers.get(block.name)
            └─ result = await handler.execute(block.input)
            └─ emit onStep("tool_result")
            └─ append tool results → continue loop

    └─ e. ITERATION COUNT CHECK → break if maxIterations reached

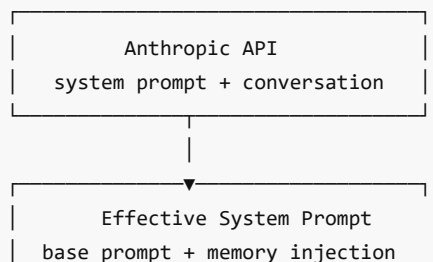
└─ 5. COST TRACKING
    └─ calculateCost(model, usage) → recordCost() via Redis

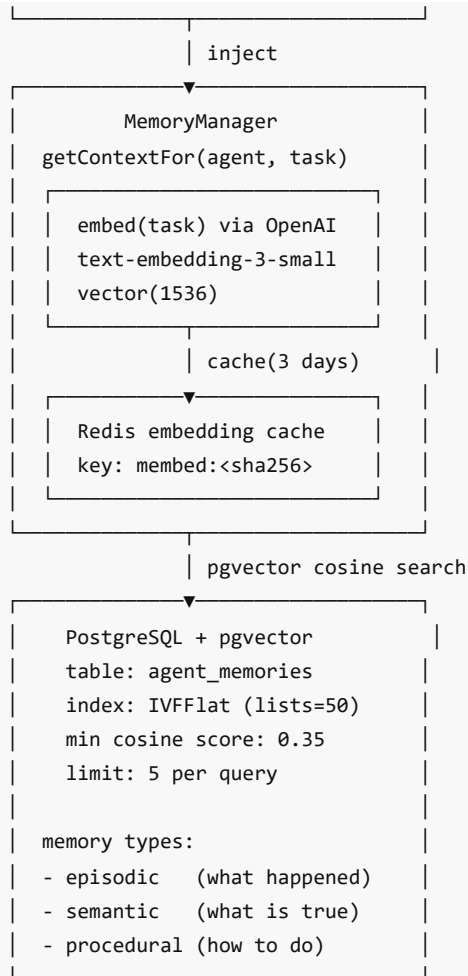
└─ 6. RESPONSE CACHE STORE (if single-turn, no tools)
    └─ responseCache.set(key, { output, model, cachedAt }, ttl)

└─ 7. MEMORY SAVE (async, non-blocking, if memorySaveEnabled)
    └─ MemoryManagerAgent.extractFromRun(agentName, userMessage, agentOutput)
        └─ Identify facts → store() in pgvector

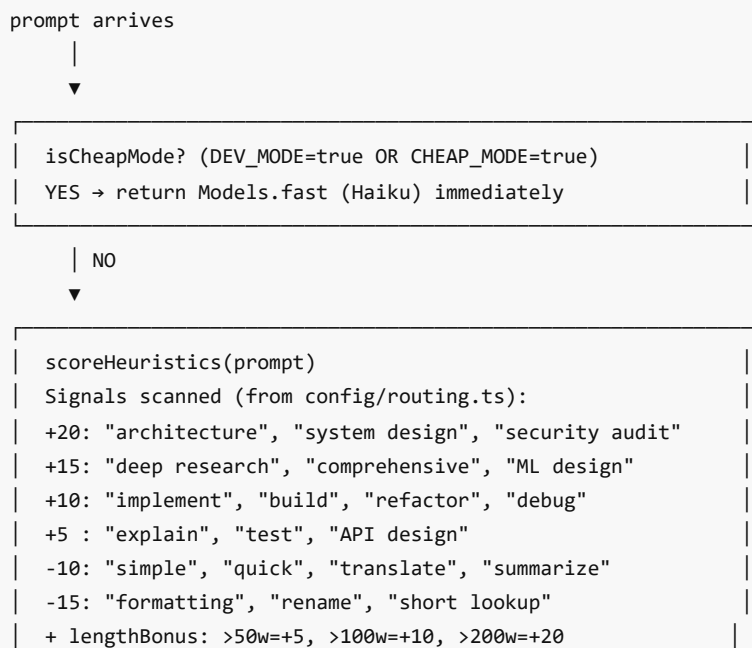
```

2.4 Diagrama de Componentes de Memória





2.5 Diagrama de Roteamento de Modelos



|
▼

```
Is decision confident?  
(score ≤ lowToMedium-15) OR (score ≥ mediumToHigh+15)  
  
YES → apply threshold:  
  score ≥ mediumToHigh → HIGH → Models.powerful (Sonnet)  
  score ≥ lowToMedium  → MEDIUM → Models.default (Haiku)  
  else                  → LOW → Models.fast (Haiku)
```

| NOT confident
▼

```
classifyWithLLM(prompt)  
→ Haiku call, max_tokens=10, responds "low"/"medium"/"high"  
→ map to tier → return appropriate model
```

3. Agentes Implementados

3.1 BaseAgent

Arquivo: `agents/_base/agent.ts`

Classe abstrata que todos os agentes especialistas estendem. Fornece:

- **Construtor:** Inicializa o cliente Anthropic, aplica configurações padrão (`Models.default` , `getMaxTokens()` , temperatura 0.3, `getMaxIterations()` , `contextTokenLimit` 80.000).
- **registerTool(handler: ToolHandler) :** Registra um tool handler — adiciona ao `toolHandlers` Map e ao array `config.tools` (schemas para a API).
- **run(userMessage, options) :** Executa o agentic loop completo (ver seção 2.3).
- **emit(onStep, step) :** Notifica step listeners — usado por scripts para imprimir progresso em tempo real.

Configuração:

```
interface AgentConfig {  
  name: string;  
  description?: string;  
  systemPrompt: string;  
  model?: string; // "auto" | model-id  
  maxTokens?: number; // default: getMaxTokens()  
  temperature?: number; // default: 0.3  
  maxIterations?: number; // default: getMaxIterations()  
  contextTokenLimit?: number; // default: 80_000  
  tools?: Anthropic.Tool[];  
  enableResponseCache?: boolean;  
  cacheTtl?: number;  
  memoryEnabled?: boolean;
```

```
memorySaveEnabled?: boolean;  
}
```

AgentResult:

```
interface AgentResult<T = string> {  
  output: T;  
  usage: {  
    inputTokens: number;  
    outputTokens: number;  
    cacheReadTokens: number;  
    cacheCreationTokens: number;  
  };  
  cost?: {  
    totalCostUsd: number;  
    savings: number;  
    breakdown: { input: number; output: number; cacheWrite: number; cacheRead: number };  
  };  
  routing?: { tier: string; model: string; score: number; reason: string };  
  fromCache: boolean;  
  contextCompressed: boolean;  
  memoriesLoaded: number;  
  memoriesSaved: number;  
  iterations: number;  
  durationMs: number;  
}
```

3.2 ResearcherAgent

Arquivo: agents/researcher/agent.ts

Propósito: Pesquisa geral com acesso à web (Tavily) e busca no vault.

Modelo padrão: Models.default (Haiku em dev, Sonnet em prod)

Tools: web_search , vault_search

System prompt: Carregado de prompts/agents/researcher.md

Uso típico:

```
tsx scripts/run-agent.ts researcher "O que são agentes AI em 2026?"  
tsx scripts/run-agent.ts researcher --model auto "Arquitetura de sistemas multi-agente"
```

3.3 CoderAgent

Arquivo: agents/coder/agent.ts

Propósito: Geração, refatoração e debugging de código TypeScript/Python.

Modelo padrão: Models.powerful (Sonnet)

Tools: code_exec , web_search

Temperatura: 0.0 (determinístico)

Uso típico:

```
tsx scripts/run-agent.ts coder "Escreva uma função que valide emails em TypeScript"
tsx scripts/run-agent.ts coder --reflect 3 "Implemente um rate limiter em Redis"
```

O flag `--reflect N` ativa o modo de reflexão: o CoderAgent gera uma resposta, o EvaluatorAgent critica, e o loop repete até N rounds ou $\text{score} \geq \text{threshold}$.

3.4 FuturecomResearcherAgent

Arquivo: `agents/futurecom-researcher/agent.ts`

Propósito: Identifica empresas-alvo em feiras enterprise brasileiras (Futurecom, ABRINT, etc.) e gera `LeadProfile` estruturados para a VRASHOWS.

Modelo padrão: `Models.powerful` (Sonnet)

Tools: `web_search`, `save_lead`

LeadProfile:

```
interface LeadProfile {
  company: string;
  segment: string; // telecom | cloud | ai | fintech | brand
  initialScore: number; // 0-100
  budgetPotential: string; // high | medium | low
  eventRelevance: string; // relevância específica para o evento
  boothComplexity: string; // complexidade esperada do stand
  website: string;
  linkedin: string;
  strategicNotes: string; // 2-3 sentences de inteligência estratégica
  sources: string[]; // URLs das fontes
}
```

Scoring de leads:

- 80-100: Presença confirmada, setor core da VRASHOWS, histórico de stands complexos
- 60-79: Setor relevante, participação provável, orçamento médio-alto
- 40-59: Participação incerta, potencial moderado
- < 40: Baixa prioridade — normalmente filtrados pelo OutreachAgent

Segmentos prioritários:

1. Telecom / Conectividade / Infraestrutura (Claro, Vivo, TIM, Embratel)
2. Cloud / SaaS / Enterprise Software (AWS, Google Cloud, Microsoft, Oracle)
3. AI / Cybersecurity (IBM, Cisco, Huawei)
4. Fintech / Bancário (Itaú, Bradesco, Nubank)
5. Brand / Varejo Premium (grandes marcas com presença recorrente)

3.5 OutreachAgent

Arquivo: `agents/outreach-agent/agent.ts`

Propósito: Gera pacotes de outreach personalizados (email + LinkedIn) para leads qualificados.

Modelo padrão: `Models.default` com `maxTokens: getMaxTokens(16384)` (extended)

maxIterations: `getMaxIterations(25)`

Tools: `memory_read` , `memory_write` , `save_outreach`

memoryEnabled: `true` (recupera outreach já gerado para deduplicação)

OutreachPackage:

```
interface OutreachPackage {
  company: string;
  leadScore: number;
  emailSubject: string;
  emailBody: string;          // 120-180 palavras
  linkedinMessage: string;    // 60-90 palavras
  meetingCta: string;         // Personalizado por empresa
  channel: "email" | "linkedin" | "both";
  tone: "consultative" | "executive" | "warm";
  segment: string;
  generatedAt: string;        // ISO timestamp
}
```

Entry points:

- `agent.generate(leads, opts, runOptions)` — batch: processa array de LeadProfile
- `agent.generateSingle(request, runOptions)` — single: processa um lead

Posicionamento no prompt (`prompts/agents/outreach-agent.md`):

- HUB de soluções integradas, não fornecedor
- Tagline: "Enquanto você fecha negócios, nós controlamos a operação"
- O que vender: tranquilidade, controle, experiência, reputação
- O que NÃO vender: staff, equipe, terceiros, serviços avulsos
- Referência obrigatória: Brasil TecPar / ABRINT 2026

3.6 LeadEnrichmentAgent

Arquivo: `agents/lead-enrichment-agent/agent.ts`

Propósito: Descobre decisores reais (nome, cargo, LinkedIn) em empresas-alvo e infere emails corporativos prováveis.

Modelo padrão: `Models.powerful` (Sonnet)

Tools: `web_search` , `resolve_email_pattern` , `save_contact`

EnrichedContact:

```
interface EnrichedContact {
  fullName: string;
  role: string;
  company: string;
  linkedinUrl?: string;
  possibleEmail?: string;      // guessedEmails[0].email
  emailInferred: boolean;      // sempre true para emails inferidos
  emailConfidence: "high" | "medium" | "low";
  guessedEmails?: GuesSEDEmail[];
  priorityScore: number;       // 0-100
  priorityReason: string;
}
```

```

    strategicNotes: string; // 1-2 sentences com contexto VRASHOWS
    outreachStatus: "pending" | "contacted" | "responded" | "disqualified";
  }

```

Prioridade de contatos:

- 80-100 (High): CMO, VP Marketing, VP Eventos, Diretores
- 50-79 (Medium): Gerentes de Marketing, Eventos, Brand, CX
- 20-49 (Low): Procurement, C-level adjacent, cargo incerto

Regras de qualidade:

- Só salvar se: nome completo (primeiro + último) + cargo confirmado + empresa
- LinkedIn e email são opcionais mas sempre pesquisados
- Nunca salvar duplicatas (mesma pessoa, mesma empresa)
- Nunca fabricar informações — marcar sempre `emailInferred: true`

3.7 EmailSenderAgent

Arquivo: `agents/email-sender-agent/agent.ts`

Propósito: Despacha emails de outreach para recipients fornecidos. Um call por recipient.

Model: `Models.default` (Haiku — tarefa determinística, sem criatividade necessária)

Tools: `send_email`

Responsabilidades:

1. Chamar `send_email` uma vez por recipient
2. Passar subject e body exatamente como fornecidos (sem reescrever)
3. Anexar media kit PDF em cold-outreach
4. Reportar status de cada envio

O agente NÃO escreve conteúdo — isso é responsabilidade do OutreachAgent. O EmailSenderAgent apenas despacha conteúdo já gerado.

Uso direto via script:

```

# Teste com email próprio
tsx scripts/run-email.ts --test-to samir@vrashows.com.br

# Com PDF anexado
tsx scripts/run-email.ts --test-to samir@vrashows.com.br \
  --attach ./assets/pdfs/vrashows_media_kit_optimized.pdf

```

3.8 MemoryManagerAgent

Arquivo: `agents/memory-manager/agent.ts`

Propósito: Extrai e persiste memórias relevantes de runs anteriores de outros agentes.

Modelo: `Models.fast` (Haiku — extração leve)

Método principal:

```

async extractFromRun(opts: {
  agentName: string;

```

```
    userMessage: string;
    agentOutput: string;
  }): Promise<Memory[]>
```

Identifica fatos, decisões e conhecimento relevante no output do agente e persiste no pgvector com tipo (episodic / semantic / procedural), importância (0.0-1.0) e tags.

3.9 EvaluatorAgent

Arquivo: agents/evaluator/agent.ts

Propósito: Avalia e critica outputs de outros agentes num loop de reflexão.

withReflection wrapper:

```
async function withReflection(
  agent: BaseAgent,
  prompt: string,
  opts: {
    maxRounds?: number; // default: 3
    scoreThreshold?: number; // default: 0.85
    onRound?: (round, eval_, output) => void;
  }
): Promise<ReflectionResult>
```

O loop:

1. Agent gera output
 2. EvaluatorAgent avalia (score 0.0-1.0) + critique textual
 3. Se score $\geq$ threshold $\rightarrow$ aceito
 4. Se score < threshold $\rightarrow$ critique injetado no próximo round como context
 5. Repete até maxRounds
-

3.10 VaultAgent e CoordinatorAgent

VaultAgent (agents/vault/agent.ts): Busca semântica no Obsidian vault local usando pgvector. Indexa notas markdown do vault via npm run vault:index . Útil para recuperar decisões arquiteturais, contexto de negócio e histórico.

CoordinatorAgent (agents/coordinator/agent.ts): Orquestrador multi-agente. Recebe tarefas complexas, decompõe em sub-tarefas, distribui para agentes especializados, agrega resultados. Suporta DAGs sequenciais e paralelos.

4. Sistema de Memória e RAG

4.1 Arquitetura de Memória em Camadas

A plataforma implementa memória em três camadas distintas:

Camada 1: Redis (short-term)
- TTL: minutos a dias

- Uso: cache de responses, dedup de email, embedding cache, cost aggregation
- Fallback: graceful no-op se Redis indisponível

- Camada 2: PostgreSQL + pgvector (long-term semantic)
- TTL: permanente (com prune aging)
 - Uso: episodic/semantic/procedural memories
 - Busca: cosine similarity (IVFFlat index)
 - Threshold: score ≥ 0.35 , importance ≥ 0

- Camada 3: Obsidian Vault (architectural knowledge)
- TTL: permanente (manual)
 - Uso: decisões arquiteturais, contexto de negócio, knowledge base semântica
 - Busca: vault_search tool via LeadEnrichmentAgent

4.2 RedisMemory — Memória de Curto Prazo

Arquivo: `memory/short-term/redis.js` (inferido de imports)

Comportamento de degradação graceful:

```
// Se Redis não estiver disponível:
// - reconnectStrategy: false → não tenta reconectar indefinidamente
// - Marca conexão como unavailable
// - Todos os get/set retornam null / ignorados silenciosamente
// - Nenhum crash no processo
```

Casos de uso:

Uso	Key prefix	TTL
Response cache	cache: + hash	30 min (low tier) / 2h (medium) / 24h (high)
Email deduplication	email:sent: + address	7 dias (604.800s)
Embedding cache	membed: + sha256[:24]	3 dias (259.200s)
Cost aggregation	agent:costs: + agentName	30 dias (2.592.000s)
Memory tool	chave arbitrária	definida pelo agente

4.3 MemoryManager — Memória de Longo Prazo

Arquivo: `memory/manager.ts`

Schema da tabela `agent_memories` :

```
CREATE TABLE agent_memories (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

type          TEXT NOT NULL CHECK (type IN ('episodic','semantic','procedural')),
content       TEXT NOT NULL,
context       TEXT NOT NULL DEFAULT '',
agent_name    TEXT NOT NULL DEFAULT '',
importance     FLOAT NOT NULL DEFAULT 0.5,    -- 0.0 a 1.0
access_count  INT NOT NULL DEFAULT 0,
tags          TEXT[] DEFAULT '{}',
embedding     vector(1536),                  -- OpenAI text-embedding-3-small
created_at    TIMESTAMPTZ DEFAULT NOW(),
last_accessed_at TIMESTAMPTZ DEFAULT NOW()
);

-- IVFFlat index para busca aproximada por cosine similarity
CREATE INDEX agent_memories_embedding_idx
  ON agent_memories USING ivfflat (embedding vector_cosine_ops)
  WITH (lists = 50);

CREATE INDEX agent_memories_agent_idx ON agent_memories (agent_name);
CREATE INDEX agent_memories_type_idx  ON agent_memories (type);

```

Tipos de memória:

- **episodic:** Fatos sobre o que aconteceu — "Na sessão de sexta, o agente gerou outreach para a Claro"
- **semantic:** Verdades gerais sobre o domínio — "A Claro prefere contato via LinkedIn para decisores acima de VP"
- **procedural:** Como fazer algo — "Para enriquecer leads AWS, usar primeiro ['firstname.lastname@amazon.com'](mailto:firstname.lastname@amazon.com)"

Busca semântica:

```

// Query embedding via OpenAI
const embedding = await openai.embeddings.create({
  model: "text-embedding-3-small",
  input: task.slice(0, 8000),    // trunca para segurança
});

// pgvector cosine search
SELECT id, content, type, importance, ...,
       1 - (embedding <=> $1::vector) AS score
FROM agent_memories
WHERE 1 - (embedding <=> $1::vector) >= 0.35
      AND importance >= 0
      [AND agent_name = $n]
      [AND type = $m]
ORDER BY score DESC
LIMIT 8;

// Após busca: atualiza access_count e last_accessed_at (fire-and-forget)

```

Consolidação de duplicatas:

```
// Encontra pares de memórias com cosine > 0.92 (DUPLICATE_THRESHOLD)
// Remove o de menor importância
await memoryManager consolidate(agentName?);

// Prune de memórias envelhecidas (importance < 0.2, access_count < 2, age > 60 days)
await memoryManager.prune(agentName?);
```

Lean Mode (ENABLE_MEMORY=false):

Quando `ENABLE_MEMORY` é definido como `"false"`, o `MemoryManager` inicializa `this.pool = null` e `this.openai = null`. Todos os métodos (`store`, `search`, `getContextFor`, etc.) retornam imediatamente com valores vazios (`"", [], { merged: 0, kept: 0, removed: 0 }`). Nenhuma exceção é lançada — o sistema opera sem infraestrutura de memória.

4.4 Injeção de Contexto de Memória

Quando `memoryEnabled: true` está configurado para um agente, antes de cada chamada à API o sistema:

1. Chama `memoryManager.getContextFor(agentName, userMessage, limit=5)`
2. Busca as 5 memórias mais relevantes com score cosine ≥ 0.40
3. Formata como:

```
## Relevant memories from past runs
- [semantic] A Claro prefere contato por email no domínio claro.com.br (importance=0.8, 3d ago)
- [episodic] Raphael Lima respondeu ao outreach em 2026-04-15 (importance=0.9, 14d ago)
```

4. Appenda ao `systemPrompt` efetivo

Este mecanismo permite que agentes "lembrem" de interações anteriores sem precisar manter contexto explícito — fundamental para deduplicação de outreach e personalização incremental.

4.5 Vault como Camada de Conhecimento

O Obsidian vault em `obsidian-vault/` atua como:

- Long-term memory arquitetural (ADRs, decisões de design)
- Semantic knowledge base (posicionamento VRASHOWS, estratégia de segmentos)
- Operational cognition layer (runbooks, playbooks)

Indexação:

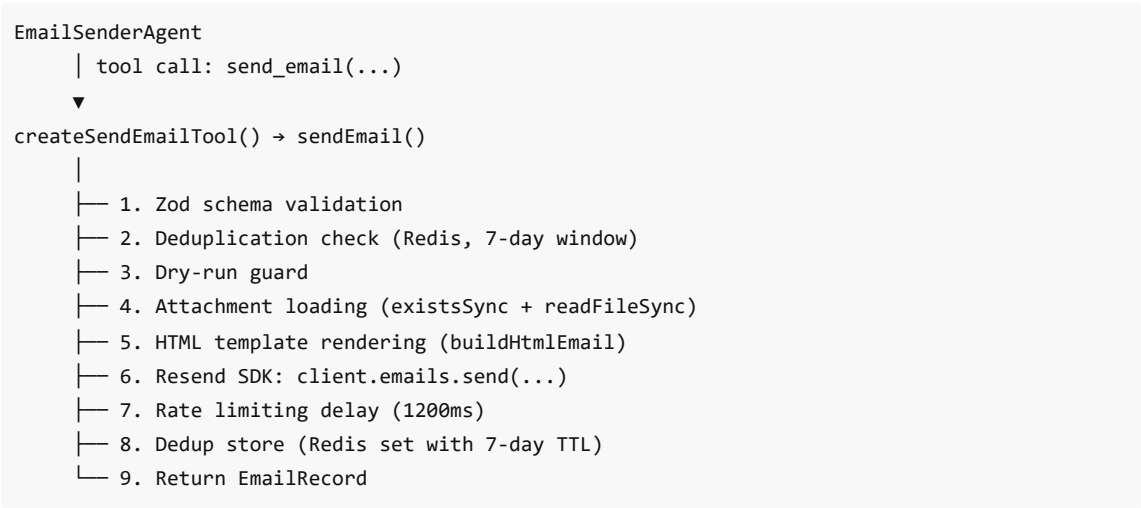
```
npm run vault:index      # indexa vault (incremental)
npm run vault:reindex    # força reindexação completa
npm run vault:stats      # estatísticas do índice
```

Busca semântica no vault:

O `VaultAgent` usa `vault_search` tool que busca no pgvector por documentos do vault indexados. Útil para o `researcher` e `coordinator` recuperarem conhecimento contextual sem precisar chamar a API para "lembrar" de decisões passadas.

5. Email Delivery System

5.1 Visão Geral do Pipeline



5.2 Resend SDK Integration

Arquivo: tools/send-email.ts

A integração com o Resend SDK (`resend ^6.4.2`) é feita através do método `client.emails.send()` :

```
const response = await client.emails.send({
  from: `${fromName} <${fromAddress}>`, // "Samir Ricardo | VRASHOWS
  <samir.ricardo@vrashows.com.br>"
  to: input.recipientEmail,
  subject: input.subject,
  text: input.bodyText, // Plain-text fallback obrigatório
  html: fullHtml, // HTML template branded VRASHOWS
  ...(attachments ? { attachments } : {}), // PDF media kit se fornecido
});
```

Domínio de envio: vrashows.com.br

From address: samir.ricardo@vrashows.com.br

From name: Samir Ricardo | VRASHOWS

DNS Records necessários para verificação no Resend:

Tipo	Host	Valor
TXT	resend._domainkey.vrashows.com.br	DKIM public key fornecida pelo Resend
MX	send.vrashows.com.br	feedback-smtp.us-east-1.amazonses.com (priority 10)
TXT	send.vrashows.com.br	v=spf1 include:amazonses.com ~all

Verificação de domínio após DNS propagação (24-48h):

```
# Após adicionar os 3 records no registrar:
curl -X POST https://api.resend.com/domains/4a323c39-5a8a-452f-8ebf-da66c11410fb/verify \
  -H "Authorization: Bearer $RESEND_API_KEY"
```

5.3 HTML Template Branded VRASHOWS

O template HTML é gerado pela função `buildHtmlEmail()` em `tools/send-email.ts`. Características:

- **Design:** Email client-compatible (Gmail, Outlook, Apple Mail)
- **Estrutura:** Header escuro (#0f172a) com nome VRASHOWS, body branco, signature, footer
- **Inline styles:** Todos os estilos inline para compatibilidade máxima (sem `<style>` externo)
- **Responsive:** `width="600"` com `meta viewport`

VRASHOWS Operações 360° Enterprise	← Header #0f172a
[CORPO DO EMAIL] Parágrafos com margin-bottom: 16px	← Body #1e293b, font-size: 15px
Samir Ricardo VRASHOWS Parcerias Estratégicas · VRASHOWS samir.ricardo@vrashows.com.br vrashows.com.br VRA	← Signature ← Link clicável #2563eb ← Link clicável #94a3b8
Você está recebendo por conta de sua atuação em [Empresa]. Para descadastrar: responda	← Footer #f8fafc

5.4 Deduplicação de Emails

O sistema previne envios duplicados via Redis:

```
const DEDUP_KEY_PREFIX = "email:sent:";
const DEFAULT_DEDUP_DAYS = 7;

// Antes de enviar:
const dedupKey = `email:sent:${input.recipientEmail}`;
const existing = await memory.get(dedupKey);
if (existing) {
  return { status: "skipped", error: `Already contacted within 7 days (last sent: ${existing})` };
}

// Após envio bem-sucedido:
const ttl = 7 * 86400; // 604.800 segundos
await memory.set(dedupKey, sentAt, ttl);
```

Se o Redis não estiver disponível, a deduplicação é silenciosamente ignorada — o email é enviado (comportamento correto: melhor enviar do que falhar).

5.5 Rate Limiting

Entre cada envio, há um delay configurável de 1200ms por padrão:

```
const DEFAULT_RATE_DELAY = 1200; // ms - abaixo do limite de 2 req/s do Resend
if (rateDelayMs > 0) {
  await new Promise<void>((r) => setTimeout(r, rateDelayMs));
}
```

Para campanhas de volume, o delay pode ser ajustado via `CoreSendOptions.rateDelayMs`. Nunca reduzir abaixo de 500ms para evitar rate limiting do Resend.

5.6 EmailRecord — Tracking de Entrega

Cada envio retorna um `EmailRecord` estruturado:

```
interface EmailRecord {
  company: string;
  contactName: string;
  recipientEmail: string;
  subject: string;
  emailType: "cold-outreach" | "follow-up" | "re-engagement";
  sequenceNumber: number; // 1=cold, 2=primeiro follow-up, 3=segundo
  sentAt: string; // ISO timestamp
  messageId: string; // Resend message ID (ou "skipped:..." / "dry:...")
  status: "sent" | "skipped" | "queued" | "failed";
  resendId?: string; // ID gerado pelo Resend para rastreabilidade
  error?: string; // Mensagem de erro se status == "failed"
}
```

Padrões de messageid:

- `"re_AbCd1234..."` — Resend ID real (status: sent)
- `"dry:email@co.com:2026-05-19T..."` — dry-run (status: queued)
- `"skipped:email@co.com:..."` — deduplicação ativa (status: skipped)
- `"failed:email@co.com:..."` — erro (status: failed)

6. Attachment System

6.1 Implementação de Anexos PDF

O suporte a anexos PDF foi implementado diretamente na função `sendEmail()` em `tools/send-email.ts`. O pipeline é:

1. Verificar existência: `existsSync(attachmentPath)`
 - └ NÃO existe → `return { status: "failed", error: "Attachment file not found: ..." }`
2. Ler arquivo: `readFileSync(attachmentPath)`

```
└─ Erro de leitura → return { status: "failed", error: "Failed to read attachment: ..." }
```

3. Converter para base64: `buffer.toString("base64")`

4. Construir array de attachments:

```
attachments = [{  
  filename: basename(attachmentPath), // "vrashows_media_kit_optimized.pdf"  
  content: base64String,  
}]
```

5. Passar ao Resend SDK:

```
client.emails.send({ ..., attachments })
```

Código completo:

```
type ResendAttachment = { filename: string; content: string };  
let attachments: ResendAttachment[] | undefined;  
  
if (input.attachmentPath) {  
  if (!existsSync(input.attachmentPath)) {  
    logger.error("[send-email] attachment not found", { path: input.attachmentPath });  
    return { ...baseRecord, status: "failed",  
      error: `Attachment file not found: ${input.attachmentPath}` };  
  }  
  try {  
    const fileBuffer = readFileSync(input.attachmentPath);  
    attachments = [{  
      filename: basename(input.attachmentPath),  
      content: fileBuffer.toString("base64"),  
    }];  
    logger.info("[send-email] attachment loaded", {  
      filename: basename(input.attachmentPath),  
      sizeKb: Math.round(fileBuffer.byteLength / 1024),  
    });  
  } catch (err) {  
    const message = err instanceof Error ? err.message : String(err);  
    return { ...baseRecord, status: "failed",  
      error: `Failed to read attachment: ${message}` };  
  }  
}
```

6.2 Media Kit Configuration

O caminho do PDF é configurado via variável de ambiente:

```
MEDIA_KIT_PDF=./assets/pdfs/vrashows_media_kit_optimized.pdf
```

No script `run-email.ts`, o padrão é:

```
const attachPath = flag("--attach") ?? (env.MEDIA_KIT_PDF ? resolve(env.MEDIA_KIT_PDF) :
```

undefined);

Localização do arquivo: assets/pdfs/vrashows_media_kit_optimized.pdf

Gitignore: assets/pdfs/ está no .gitignore — o PDF nunca é commitado ao repositório.

6.3 Regras de Anexo por Tipo de Email

Tipo de Email	Anexar PDF?	Motivo
cold-outreach (sequenceNumber=1)	Sempre	Primeira impressão — o PDF contextualiza a VRASHOWS
follow-up (sequenceNumber=2,3)	Nunca	Já foi enviado no cold outreach
re-engagement	Apenas se houve resposta anterior sem conversão	Renova o contexto visual
executive-intro	Apenas se há evento específico iminente	Tom peer-to-peer não inclui pitch completo

7. Lead Enrichment System

7.1 EmailPatternResolver

Arquivo: agents/lead-enrichment-agent/email-resolver.ts

O EmailPatternResolver é um sistema puro de inferência heurística de emails corporativos — sem chamadas externas, sem scraping, sem APIs. Funciona inteiramente com:

- 1. **Company domain registry** (40+ empresas mapeadas)
- 2. **Portuguese name normalization** (accents + articles)
- 3. **Email pattern generation** (6 padrões por contato)
- 4. **Confidence scoring** (high/medium/low)

Resolução em 3 níveis:

Nível 1: Company registry (KNOWN_DOMAINS)
"aws" → { domain: "amazon.com", primaryPattern: "firstname.lastname", confidence: "high" }

Nível 2: Website URL parsing
"https://minha-empresa.com.br" → domain: "minha-empresa.com.br", confidence: "medium"

Nível 3: Company name slug inference
"Empresa XYZ" → domain: "empresaxyz.com.br", confidence: "low"

Registro de empresas (KNOWN_DOMAINS) — seleção:

Chave	Domínio	Padrão primário	Confiança
aws / amazon web services	amazon.com	firstname.lastname	high
google	google.com	firstname	medium

microsoft / azure	microsoft.com	firstname.lastname	medium
oracle	oracle.com	firstname.lastname	high
salesforce	salesforce.com	firstname.lastname	high
ericsson	ericsson.com	firstname.lastname	high
huawei	huawei.com	firstname.lastname	high
claro	claro.com.br	firstname.lastname	high
vivo / telefonica	vivo.com.br / telefonica.com.br	firstname.lastname	high
tim	tim.com.br	firstname.lastname	high
nubank	nubank.com.br	firstname.lastname	high
totvs	totvs.com	firstname.lastname	high
stefanini	stefanini.com	firstname.lastname	high
bradesco	bradesco.com.br	flastname	medium
cisco	cisco.com	flastname	medium

6 padrões de email gerados:

Padrão	Exemplo (João da Silva)
firstname.lastname	joao.silva@amazon.com
flastname	jsilva@amazon.com
f.lastname	j.silva@amazon.com
firstname	joao@amazon.com
firstname_lastname	joao_silva@amazon.com
firstnamelastname	joaosilva@amazon.com

7.2 Normalização de Nomes em Português

```
// Mapa de acentos para ASCII
const ACCENT_MAP = {
  á:"a", à:"a", â:"a", ã:"a", ä:"a",
  é:"e", è:"e", ê:"e", ë:"e",
  í:"i", ì:"i", î:"i", ï:"i",
  ó:"o", ò:"o", ô:"o", õ:"o", ö:"o",
  ú:"u", ù:"u", û:"u", ü:"u",
  ç:"c", ñ:"n",
};

// Artigos/preposições portuguesas removidos de sobrenomes compostos
```

```
const PT_ARTICLES = new Set(["da","de","do","das","dos","di","e","del"]);

// Exemplos de normalização:
// "João da Silva"      → first="joao", last="silva"
// "Maria Clara Nunes"  → first="maria", last="nunes"
// "Aline de Oliveira"  → first="aline", last="oliveira"
// "Rafael Ávila"       → first="rafael", last="avila"
```

7.3 Workflow de Enriquecimento

Para cada empresa-alvo, o `LeadEnrichmentAgent` executa:

```
Para cada empresa:
|
|— 1. WEB SEARCH (múltiplas queries)
|   |— "[Company] diretor marketing eventos linkedin"
|   |— "[Company] gerente eventos corporativos"
|   |— "[Company] head of events marketing"
|   |— "site:linkedin.com/in [Company] marketing eventos"
|   |— "[Company] patrocínio Futurecom [year]"
|   |— "[Company] CMO OR 'VP Marketing' OR 'Diretor de Marketing'"
|
|— 2. VALIDATE CONTACTS (cross-reference results)
|   |— Confirmar nome completo + cargo + empresa
|
|— 3. resolve_email_pattern (para cada contato)
|   |— Input: { name, company, website? }
|   |— Output: { domain, guessedEmails[], confidence, reasoning }
|
|— 4. save_contact (para cada contato validado)
|   |— Input: EnrichedContact + guessedEmails[]
|   |— Validates via Zod schema
```

7.4 Leads AWS LATAM — Exemplo Real

O arquivo `data/leads/aws-leads.json` contém 6 decisores identificados para a VRASHOWS:

Nome	Cargo	Score	Email Inferido	Confiança
Rachel Louise Wilson	Regional Director, Enterprise	91	rachel.wilson@amazon.com	high
Raphael Lima	Head of Marketing LATAM	88	raphael.lima@amazon.com	high
Aishwarya Murali	Senior Manager, Events & Experiences	85	aishwarya.murali@amazon.com	high
Jayme Faria	Director, Partner & Alliance Marketing	82	jayme.faria@amazon.com	high
Marcio Pitel	Head of Sales, Telecom LATAM	78	marcio.pitel@amazon.com	high
Takashi Sato	Solutions Architect Manager	58	takashi.sato@amazon.com	high

Todos com `outreachStatus: "pending"` e arrays `guessedEmails[]` com 6 variantes cada.

8. Outbound Strategy

8.1 Posicionamento VRASHOWS

A VRASHOWS deve ser posicionada **sempre como parceiro estratégico operacional**, nunca como fornecedor de serviços.

Identidade central:

HUB premium de soluções integradas para eventos corporativos e experiências de marca.
Tagline: "Enquanto você fecha negócios, nós controlamos a operação."

O que vender (benefícios reais):

Benefício	Descrição
Tranquilidade	Time de liderança foca 100% em negócios e relacionamentos
Controle	Operação integrada com visibilidade em tempo real
Experiência	Visitantes, parceiros e clientes com experiência memorável
Reputação	Presença de marca de nível internacional, sem improvisos
Parceria	Único ponto de contato para toda a operação do evento

O que NÃO vender:

- Staff (parece terceirização)
- Equipe (despersonaliza)
- Terceiros (gera insegurança)
- Serviços avulsos (fragmenta o valor)
- Preços ou tabelas (vai para proposta, não para email)

8.2 Templates de Outreach

Localização: `assets/templates/`

8.2.1 Cold Outreach (`cold-outreach.md`)

Tom: Consultivo, peer-to-peer, 120-180 palavras.

Attachment: PDF media kit obrigatório.

Estrutura:

1. Abertura (1 linha) — referência específica ao segmento da empresa
2. HUB identity (2-3 linhas) — o que a VRASHOWS é
3. Case ABRINT (1 linha) — Brasil TecPar como credencial
4. Tagline (implícita ou explícita)
5. CTA (20 minutos para conversa)

8.2.2 Follow-up (`follow-up.md`)

Uso: 3-7 dias após cold outreach sem resposta.

Tom: Levíssimo, sem pressão.

Attachment: Nunca re-enviar PDF.

3 variantes:

- Variante 1 (~70 palavras): Referência ao case ABRINT
- Variante 2 (~60 palavras): Calendário + urgência suave
- Variante 3 (~25 palavras): Ultra-curto — "Seguindo meu email anterior..."

Regras: Máx. 80 palavras. Nunca: "última chance", "só essa semana".

8.2.3 Re-engagement (re-engagement.md)

Uso: Após ciclo completo (cold + 2 follow-ups) ou inativos há 30+ dias.

Tom: Renovar interesse com contexto novo. Nunca parecer insistente.

Attachment: Reenviar PDF apenas se houve resposta anterior sem conversão.

2 variantes:

- Variante 1 (~80 palavras): Gatilho de evento
- Variante 2 (~70 palavras): Nova capability VRASHOWS

Regra crítica: Se não houver resposta ao re-engagement → marcar como `disqualified`.

Nunca: "tentei entrar em contato várias vezes", "última tentativa".

8.2.4 Executive Intro (executive-intro.md)

Uso: Contatos C-level (CMO, VP, Country Manager, CEO).

Tom: Peer-to-peer entre executivos. Ultra-conciso. Foco em parceria estratégica.

Attachment: PDF opcional — incluir apenas se há evento específico e iminente.

Regras: Subject máx. 8 palavras. Body máx. 100 palavras. CTA: "20 minutos".

Nunca: "adoraria conectar", "fique à vontade", "qualquer dúvida", preços, tabelas.

8.3 Sequência de Outreach Recomendada

Dia 1:	Cold outreach + PDF media kit
Dia 3:	Follow-up (Variante 1 – case reference)
Dia 7:	Follow-up (Variante 3 – ultra-curto) └ Sem resposta: marcar como "awaiting" por 30 dias
Dia 37:	Re-engagement (Variante 1 ou 2 – gatilho novo) └ Sem resposta: marcar como "disqualified"

8.4 Segmentação por Indústria

Segmento	Posicionamento VRASHOWS	Tom	CTA
Telecom / Conectividade	Confiabilidade operacional + credibilidade de marca em escala	Técnico + estratégico	"Quero conhecer como operamos para [Empresa] no [Evento]"
Cloud / SaaS Enterprise	Experiência premium refletindo a promessa da marca	Inovação + premium	"20 minutos para alinhar como traduzimos sua marca no evento"
AI / Cybersecurity	Ambientes de precisão + confiança executive-level	Confiança + controle	"Conversa rápida sobre operação para [Evento]?"

Fintech	Solidez institucional + hospitalidade compliance-grade	Institucional	"Alinhamento de 20 minutos sobre presença em [Evento]"
Brand / Varejo Premium	Design de experiência imersivo + ROI de marca	Aspiracional	"Podemos criar algo único para [Empresa] em [Evento]?"

9. Cost Optimization System

9.1 Pricing de Modelos

Arquivo: `config/costs.ts`

```
export const MODEL_PRICING = {
  "claude-haiku-4-5-20251001": {
    input:      $0.80 / 1M tokens,
    output:     $4.00 / 1M tokens,
    cacheWrite: $1.00 / 1M tokens,
    cacheRead:  $0.08 / 1M tokens, // 10x cheaper than input
  },
  "claude-sonnet-4-6": {
    input:      $3.00 / 1M tokens,
    output:     $15.00 / 1M tokens,
    cacheWrite: $3.75 / 1M tokens,
    cacheRead:  $0.30 / 1M tokens, // 10x cheaper than input
  },
  "claude-opus-4-7": {
    input:      $15.00 / 1M tokens,
    output:     $75.00 / 1M tokens,
    cacheWrite: $18.75 / 1M tokens,
    cacheRead:  $1.50 / 1M tokens, // 10x cheaper than input
  },
};
```

9.2 Modos de Custo

Tabela de impacto — Dev Mode vs. Production:

Parâmetro	Dev/Cheap Mode	Production
Modelo padrão	Haiku (\$0.80/1M input)	Sonnet (\$3.00/1M input)
Max output tokens	2.048	8.192 (default) / 16.384 (extended)
Max iterations	5	10 (default) / 25 (outreach)
Model router	Bypass → sempre Haiku	Heuristic + LLM classifier ativo
Custo estimado p/ run simples	~\$0.001	~\$0.01 – \$0.05

Ativação do Cheap Mode:

```
DEV_MODE=true
CHEAP_MODE=true
MAX_TOOL_ITERATIONS=5
MAX_OUTPUT_TOKENS=2048
```

Ativação do Production Mode:

```
DEV_MODE=false
CHEAP_MODE=false
# MAX_TOOL_ITERATIONS não definido → usa padrão do agente
# MAX_OUTPUT_TOKENS não definido → usa padrão do agente
```

9.3 Prompt Caching

Todos os agentes enviam o system prompt com `cache_control: { type: "ephemeral" }` :

```
system: [{
  type: "text",
  text: effectiveSystemPrompt,
  cache_control: { type: "ephemeral" },
}],
```

Isso instrui a Anthropic API a cachear o system prompt no primeiro uso. Em runs subsequentes com o mesmo prompt:

- **Cache write:** ~25% mais caro que input normal (one-time cost)
- **Cache read:** ~10× mais barato que input normal (recorrente)

Break-even: A partir da 2ª chamada com o mesmo system prompt, o caching é lucrativo.

Tracking de savings:

```
export interface CostBreakdown {
  totalCost: number;
  wouldHaveCost: number; // sem nenhum caching
  savings: number;       // wouldHaveCost - totalCost
}
```

9.4 Token Caps Dinâmicos

```
// getMaxTokens() em config/models.ts:
export function getMaxTokens(preferred?: number): number {
  const base = preferred ?? ModelConfig.maxTokens.default; // 8192
  if (env.MAX_OUTPUT_TOKENS) return Math.min(base, env.MAX_OUTPUT_TOKENS);
  if (isCheapMode) return Math.min(base, ModelConfig.maxTokens.cheap); // 2048
  return base;
}

// getMaxIterations() em config/models.ts:
```

```
export function getMaxIterations(preferred?: number): number {
  const base = preferred ?? (isCheapMode ? 5 : 10);
  if (env.MAX_TOOL_ITERATIONS) return Math.min(base, env.MAX_TOOL_ITERATIONS);
  return base;
}
```

Lógica: O agente pode solicitar um limite preferido (ex: `getMaxTokens(16384)` para `extended`), mas o `environment cap` sempre prevalece via `Math.min`. O `Cheap Mode` aplica caps mais baixos automaticamente mesmo sem configuração explícita no `.env`.

9.5 Cost Tracking por Agente

Todos os runs de agentes registram custo no Redis:

```
// Chave: "agent:costs:outreach-agent"
// TTL: 30 dias
interface AgentCostRecord {
  agent: string;
  model: string;
  totalCostUsd: number;
  totalInputTokens: number;
  totalOutputTokens: number;
  totalCacheReadTokens: number;
  totalCacheCreationTokens: number;
  totalSavingsUsd: number;
  runs: number;
  lastRunAt: string;
}
```

Relatório de custos:

```
npm run costs
# ou
tsx scripts/cost-report.ts
```

9.6 Estratégia de Roteamento por Custo

Tier	Modelo	Quando usar
LOW	Haiku	Formatação, tradução, sumário, lookups simples, email dispatch
MEDIUM	Haiku / Sonnet	Coding, debugging, API design, análise moderada
HIGH	Sonnet	Arquitetura de sistema, pesquisa profunda, auditorias, rewrites

Nota: `Opus ( claude-opus-4-7 )` não está ativamente roteado no sistema atual — reservado para tarefas futuras de planejamento/reflexão de alto nível.

10. Configuração Completa

10.1 Pré-requisitos

Requisito	Versão mínima	Verificação
Node.js	22.x	<code>node --version</code>
npm	10.x	<code>npm --version</code>
Docker Desktop	4.x	<code>docker --version</code>
Git	2.x	<code>git --version</code>

API Keys necessárias:

- **Anthropic API Key** — obrigatória (`ANTHROPIC_API_KEY`)
- **Resend API Key** — para envio de emails (`RESEND_API_KEY`)
- **OpenAI API Key** — para embeddings/memória (`OPENAI_API_KEY`)
- **Tavily API Key** — para web search (`TAVILY_API_KEY`)

10.2 Instalação Passo a Passo

```
# 1. Clonar repositório
git clone <repo-url> ai-cognitive-runtime
cd ai-cognitive-runtime

# 2. Instalar dependências
npm install

# 3. Configurar variáveis de ambiente
cp .env.example .env
# Editar .env com suas chaves de API (ver seção 10.3)

# 4. Subir infraestrutura (Redis + PostgreSQL/pgvector)
npm run infra:up

# 5. Verificar containers
docker ps
# Esperado:
# ai-lab-redis      (porta 6379)
# ai-lab-postgres   (porta 5433)

# 6. Verificar type check (sem build step necessário)
npm run typecheck
# Esperado: zero erros

# 7. Teste rápido do agente researcher
tsx scripts/run-agent.ts researcher "Explique o que são agentes AI"

# 8. Teste de email (com email próprio)
tsx scripts/run-email.ts --test-to seu@email.com
```

10.3 Variáveis de Ambiente — Referência Completa

Arquivo: `config/env.ts` — validado com Zod schema no startup.

```
# — API Keys (obrigatórias) —————
ANTHROPIC_API_KEY=sk-ant-api03-... # Obrigatório

# — API Keys (opcionais) —————
OPENAI_API_KEY=sk-proj-... # Embeddings + vault search
TAVILY_API_KEY=tvly-... # Web search tool
RESEND_API_KEY=re-... # Email delivery
RESEND_FROM_EMAIL=samir@vrashows.com.br # Sender address
RESEND_FROM_NAME=Samir Ricardo | VRASHOWS # Sender name

# — Modelos —————
DEFAULT_MODEL=claude-haiku-4-5-20251001 # Padrão para a maioria dos agentes
FAST_MODEL=claude-haiku-4-5-20251001 # Usado pelo router em tasks leves
POWERFUL_MODEL=claude-sonnet-4-6 # Para tasks complexas

# — Infraestrutura —————
REDIS_URL=redis://localhost:6379
DATABASE_URL=postgres://ailab:ailab@localhost:5433/ai_lab
VAULT_PATH=~/.obsidian-vault

# — Memória —————
ENABLE_MEMORY=false # "false" = Lean Mode (sem Postgres/Redis)
MEMORY_PROVIDER=local # Futuro: "supabase" | "neon"

# — Observabilidade —————
LOG_LEVEL=info # debug | info | warn | error

# — Cost / Dev mode —————
DEV_MODE=true # Haiku default, caps reduzidos
CHEAP_MODE=true # Alias de DEV_MODE para cost control
MAX_TOOL_ITERATIONS=5 # Cap de iterações por run (default: 10)
MAX_OUTPUT_TOKENS=2048 # Cap de output tokens (default: 8192)

# — Assets —————
MEDIA_KIT_PDF=./assets/pdfs/vrashows_media_kit_optimized.pdf
```

Valores derivados (computed em env.ts):

```
export const isCheapMode = env.CHEAP_MODE === "true" || env.DEV_MODE === "true";
```

10.4 Docker Compose — Infraestrutura

Arquivo: `docker-compose.yml`

```
services:
  postgres:
    image: pgvector/pgvector:pg16 # PostgreSQL 16 com extensão vector pré-instalada
    container_name: ai-lab-postgres
```

```

environment:
  POSTGRES_USER: ailab
  POSTGRES_PASSWORD: ailab
  POSTGRES_DB: ai_lab
ports:
  - "5433:5432" # Porta host: 5433 (evita conflito com Postgres
local)
volumes:
  - postgres_data:/var/lib/postgresql/data
  - ./infra/postgres/init.sql:/docker-entrypoint-initdb.d/init.sql
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U ailab -d ai_lab"]
  interval: 5s; timeout: 5s; retries: 5

redis:
  image: redis:7-alpine
  container_name: ai-lab-redis
  ports:
    - "6379:6379"
  volumes:
    - redis_data:/data
  command: redis-server --appendonly yes # Persistência AOF
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 5s; timeout: 3s; retries: 5

```

Comandos de infraestrutura:

```

npm run infra:up      # docker compose up -d
npm run infra:down    # docker compose down
npm run infra:reset   # docker compose down -v && docker compose up -d (wipe volumes)
npm run infra:logs    # docker compose logs -f (tail em tempo real)

```

10.5 tsconfig.json

```

{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "strict": true,
    "outDir": "./dist",
    "rootDir": ".",
    "skipLibCheck": true
  }
}

```

Nota: `tsx` executa TypeScript diretamente sem build step. Use `npm run typecheck` (= `tsc --noEmit`) para validar tipos sem gerar arquivos.

11. Execução dos Agentes

11.1 CLI Principal — run-agent.ts

```
tsx scripts/run-agent.ts <agent> [--model auto|<model-id>] [--reflect [n]] "<prompt>"
```

Agentes disponíveis:

Nome	Descrição
researcher	Pesquisa geral com web search
coder	Geração e debugging de código
vault	Busca semântica no Obsidian vault
memory-manager	Extração e manutenção de memórias
futurecom-researcher	Pesquisa de leads em feiras enterprise
outreach-agent	Geração de pacotes de outreach
lead-enrichment-agent	Enriquecimento de contatos e inferência de emails
email-sender-agent	Envio de emails via Resend

Exemplos reais:

```
# Pesquisa simples (Haiku por default em DEV_MODE)
tsx scripts/run-agent.ts researcher "Quais empresas confirmaram presença no Futurecom 2026?"

# Pesquisa com modelo específico
tsx scripts/run-agent.ts researcher --model claude-sonnet-4-6 "Análise competitiva de hubs de eventos enterprise no Brasil"

# Roteamento automático (detecta complexidade e escolhe modelo)
tsx scripts/run-agent.ts coder --model auto "Implemente um parser de CSV com tratamento de erros robusto"

# Modo reflexão (3 rounds de critique)
tsx scripts/run-agent.ts coder --reflect 3 "Escreva e refine uma implementação de retry com exponential backoff"

# Lead enrichment para empresa específica
tsx scripts/run-agent.ts lead-enrichment-agent "Enriqueça os decisores da Claro Brasil para outreach VRASHOWS"

# Outreach batch
tsx scripts/run-agent.ts outreach-agent "Gere pacotes de outreach para os leads da AWS LATAM no arquivo data/leads/aws-leads.json"

# Email sender (com agent)
```

```
tsx scripts/run-agent.ts email-sender-agent "Envie os emails de outreach para os contatos AWS com score >= 80"
```

Output do run:

```
[router] heuristic → tier=low model=claude-haiku-4-5-20251001

[tool] web_search({"query":"Futurecom 2026 empresas confirmadas"...})

--- OUTPUT ---

As principais empresas confirmadas no Futurecom 2026 incluem...

[routing] tier=low model=claude-haiku-4-5-20251001 score=-5
[cost]     $0.000847 saved=$0.000021
[tokens]   in:1247 out:389 cache_read:982 cache_write:0
```

11.2 CLI de Email — run-email.ts

```
tsx scripts/run-email.ts [--test-to <email>] [--attach <path>] [--dry-run]
```

Flags:

Flag	Descrição
--test-to <email>	Envia email de teste para o endereço especificado
--attach <path>	Caminho do PDF a anexar (override de MEDIA_KIT_PDF)
--dry-run	Simula o envio sem chamar o Resend (status: "queued")

Exemplos:

```
# Teste básico
tsx scripts/run-email.ts --test-to samir.ricardo@vrashows.com.br

# Com PDF media kit
tsx scripts/run-email.ts --test-to samir@email.com \
  --attach ./assets/pdfs/vrashows_media_kit_optimized.pdf

# Dry run (não envia, apenas valida)
tsx scripts/run-email.ts --test-to samir@email.com --dry-run
```

11.3 Modo de Reflexão — withReflection

O `EvaluatorAgent` pode ser encadeado com qualquer agente para um loop de auto-melhoria:

```
import { withReflection } from './agents/evaluator/agent.js';

const result = await withReflection(agent, prompt, {
```

```

maxRounds: 3,          // máximo de rounds de critique
scoreThreshold: 0.85,  // aceitar se score >= 0.85
onRound: (round, evaluation, output) => {
  console.log(`Round ${round}: score=${evaluation.score.toFixed(2)},
passed=${evaluation.passed}`);
  if (!evaluation.passed) console.log(`Critique: ${evaluation.critique}`);
},
});

console.log(`Final output: ${result.output}`);
console.log(`Rounds used: ${result.rounds}, passed: ${result.passed}`);
console.log(`Total cost: $$${result.totalCostUsd.toFixed(6)}`);

```

Output no CLI:

```

[reflect] enabled – up to 3 round(s)

[reflect] round=1 score=0.72 passed=false
        critique: Email body lacks specific reference to company segment...
[reflect] round=2 score=0.91 passed=true

--- OUTPUT ---
[conteúdo melhorado]

[reflect] rounds=2 passed=true score=0.91
[cost]      $0.003421

```

11.4 Relatório de Custos

```

npm run costs
# ou
tsx scripts/cost-report.ts

```

Consulta o Redis e imprime um relatório de custo acumulado por agente:

AI Lab – Cost Report			
Agent	Total USD	Runs	Saved USD
outreach-agent	\$0.0847	12	\$0.0231
lead-enrichment-agent	\$0.0341	8	\$0.0089
email-sender-agent	\$0.0012	24	\$0.0003
researcher	\$0.0234	31	\$0.0062

11.5 Manutenção de Memória

```

npm run memory          # Estatísticas de memória

```

```
npm run memory:maintain    # Consolidate (dedup) + prune (aging)
```

Via código:

```
import { memoryManager } from "../memory/manager.js";

await memoryManager.initialize();

// Ver estatísticas
const stats = await memoryManager.stats("outreach-agent");
console.log(stats);
// { byType: [{ type: "episodic", count: 14, avg_importance: 0.72 }, ...] }

// Consolidar memórias duplicadas (cosine > 0.92)
const result = await memoryManager consolidate("outreach-agent");
console.log(result);
// { merged: 3, kept: 11, removed: 3 }

// Remover memórias envelhecidas (importance < 0.2, access < 2, age > 60d)
const pruned = await memoryManager.prune("outreach-agent");
console.log(`Pruned: ${pruned} memories`);
```

12. Git e Versionamento

12.1 Estrutura de Branches

main	← branch principal de produção
feat/*	← novas funcionalidades
fix/*	← correções
refactor/*	← refatorações (sem mudança de comportamento)
docs/*	← documentação

12.2 O que NUNCA Commitar

```
# Arquivo .gitignore atual:

# Secrets – CRÍTICO
.env
.env.local
.env.*.local

# Dependências
node_modules/

# Build (não usado em dev, mas por completude)
dist/
coverage/

# Logs e artefatos gerados
```

```
logs/
*.log
data/outreach/

# PDF do media kit (confidencial)
assets/pdfs/

# Arquivos temporários
*.tmp
*.temp

# OS
.DS_Store
Thumbs.db

# IDEs
.vscode/
.idea/
```

Verificação antes de commit:

```
git status
git diff --cached
```

Se um secret for commitado acidentalmente:

```
# 1. Invalidar a key IMEDIATAMENTE no dashboard da API (Anthropic, Resend, etc.)
# 2. Remover do histórico com git filter-repo ou BFG Repo-Cleaner
# 3. Force push (coordenar com equipe)
# 4. Gerar nova key

# Verificar se secrets estão rastreados:
git log --all --full-history -- .env
```

12.3 Padrão de Commit

Commits seguem Conventional Commits:

```
<type>(<scope>): <description>
```

```
<body opcional>
```

```
Co-Authored-By: Claude Sonnet 4.6 <noreply@anthropic.com>
```

Types:

- feat: — nova funcionalidade
- fix: — correção de bug
- refactor: — sem mudança de comportamento
- docs: — documentação

- chore: — manutenção (deps, config)

Exemplos reais do projeto:

```
fix: remove secrets and node_modules from repository
feat: add PDF attachment support to email-sender-agent
feat: implement EmailPatternResolver for corporate email inference
feat: add VRASHOWS HUB positioning to all agent prompts and templates
refactor: extract getMaxTokens/getMaxIterations for cost control
```

12.4 Workflow de Desenvolvimento

```
# 1. Status antes de começar
git status

# 2. Criar branch para a feature
git checkout -b feat/lead-enrichment-phase-2

# 3. Desenvolver com tsx (sem build step)
tsx scripts/run-agent.ts researcher "test"

# 4. Type check antes de commitar
npm run typecheck

# 5. Stage apenas arquivos relevantes (nunca git add -A com .env presente)
git add agents/lead-enrichment-agent/
git add prompts/agents/lead-enrichment-agent.md

# 6. Commit com mensagem descritiva
git commit -m "feat(enrichment): add email pattern resolver with 40+ company registry"

# 7. Push e PR
git push origin feat/lead-enrichment-phase-2
```

12.5 Problemas Comuns com Git no Windows/PowerShell

Commit com mensagem multiline em PowerShell:

```
# Método correto: usar here-string PowerShell
git commit -m @"
feat: add comprehensive email enrichment system

Implements EmailPatternResolver with 40+ company registry,
Portuguese name normalization, and 6-pattern variant generation.

Co-Authored-By: Claude Sonnet 4.6 <noreply@anthropic.com>
"@

# OU: escrever em arquivo temporário
Set-Content -Path ".git_commit_msg.txt" -Encoding utf8 -Value @"
feat: add comprehensive email enrichment system
```

```
Implements EmailPatternResolver with 40+ company registry.
```

```
Co-Authored-By: Claude Sonnet 4.6 <noreply@anthropic.com>
```

```
"@"
```

```
git commit -F .git_commit_msg.txt
```

```
Remove-Item .git_commit_msg.txt
```

NOTA: Em PowerShell, `<<'EOF'` (bash heredoc) NÃO funciona. Use `@'...'@` (here-string de aspas simples) para strings literais sem interpolação.

13. Troubleshooting Completo

13.1 Erros de Startup (Validação de Env)

Problema: Invalid environment variables: { ANTHROPIC_API_KEY: ['Required'] }

Causa: `.env` não existe ou está incompleto.

Solução:

```
# Verificar se .env existe
ls -la .env

# Se não existir:
cp .env.example .env

# Editar .env com as keys reais
```

Problema: RESEND_FROM_EMAIL: ['Invalid email']

Causa: Email no `.env` está mal formatado.

Solução:

```
# Errado:
RESEND_FROM_EMAIL=samir.ricardo@vrashows

# Correto:
RESEND_FROM_EMAIL=samir.ricardo@vrashows.com.br
```

Problema: DATABASE_URL required for MemoryManager

Causa: `ENABLE_MEMORY` não está `"false"` e `DATABASE_URL` está ausente.

Solução (Lean Mode):

```
ENABLE_MEMORY=false
```

Solução (Full Mode):

```
ENABLE_MEMORY=true
DATABASE_URL=postgresql://ailab:ailab@localhost:5433/ai_lab
OPENAI_API_KEY=sk-proj-...
```

13.2 Erros de Infraestrutura (Docker)

Problema: Error: connect ECONNREFUSED 127.0.0.1:6379 (Redis)

Diagnóstico:

```
docker ps | grep redis      # Verificar se container está rodando
docker logs ai-lab-redis    # Ver logs do Redis
```

Solução:

```
npm run infra:up
# Aguardar healthcheck: redis-cli ping → PONG
```

Problema: Error: connect ECONNREFUSED 127.0.0.1:5433 (PostgreSQL)

Diagnóstico:

```
docker ps | grep postgres
docker logs ai-lab-postgres
```

Solução:

```
npm run infra:up
# Aguardar healthcheck: pg_isready -U ailab -d ai_lab
```

Problema: PostgreSQL já existe com schema desatualizado.

Solução (wipe e restart):

```
npm run infra:reset    # ATENÇÃO: apaga todos os dados
```

13.3 Erros de TypeScript

Problema: Type 'string' is not assignable to type '"high" | "medium"'

Causa: Literal type mismatch. Exemplo: `patternConfidence: "low"` quando o tipo aceita apenas `"high" | "medium"`.

Solução: Verificar o tipo esperado e usar o valor correto:

```
// Errado:
{ patternConfidence: "low" }
```

```
// Correto:
{ patternConfidence: "medium" }
```

Problema: Cannot find module './email-resolver.js'

Causa: No ESM com TypeScript, importações devem usar extensão `.js` mesmo para arquivos `.ts`.

Solução:

```
// Errado:
import { emailPatternResolver } from "./email-resolver";

// Correto:
import { emailPatternResolver } from "./email-resolver.js";
```

Problema: SyntaxError: Cannot use import statement in a module

Causa: Arquivo `.ts` sendo executado com `node` diretamente (sem `tsx`).

Solução:

```
# Errado:
node scripts/run-agent.ts

# Correto:
tsx scripts/run-agent.ts
# ou
npx tsx scripts/run-agent.ts
```

13.4 Erros de Email (Resend)

Problema: You can only send testing emails to your own email address

Causa: Domínio de envio (`vrashows.com.br`) não está verificado no Resend. Em modo não verificado, o Resend só permite envios para o email da conta registrada.

Solução:

1. Verificar o domínio (ver seção 5.2)
2. Ou usar o email da conta Resend como remetente temporário

Problema: Attachment file not found: ./assets/pdfs/vrashows_media_kit_optimized.pdf

Causa: O arquivo PDF não existe no caminho especificado.

Solução:

```
# Verificar existência
ls assets/pdfs/

# Colocar o PDF no diretório correto
# (o diretório está em .gitignore – precisa ser adicionado manualmente)
```

```
mkdir -p assets/pdfs
# Copiar o PDF para assets/pdfs/vrashows_media_kit_optimized.pdf
```

Problema: Email enviado sem formatação HTML (só texto plano)

Causa: `bodyHtml` não fornecido, ou o cliente de email do recipient não suporta HTML.

Solução: O sistema sempre envia tanto `text` quanto `html`. Se o recipient vê só texto, é configuração do cliente de email dele — comportamento esperado.

Problema: Rate limit error do Resend

Causa: Muitos emails em sequência rápida.

Solução: O sistema aplica automaticamente 1200ms de delay entre envios. Se persistir, aumentar o delay:

```
// Em CoreSendOptions:
const opts: CoreSendOptions = {
  rateDelayMs: 2000, // aumentar para 2s
};
```

13.5 Erros do Agentic Loop

Problema: Agente atinge `maxIterations` sem completar a tarefa

Diagnóstico:

```
[cost] $0.0234 saved=$0.0012
[tokens] in:12847 out:2389 cache_read:9832 cache_write:0
```

Nota-se muitos tokens sem output final.

Causas comuns:

1. Prompt muito vago → agente fica em loop de tool calls
2. Tool retornando erros continuamente → agente tenta recuperar
3. `maxIterations` muito baixo para a complexidade da tarefa

Soluções:

```
# Aumentar iterations pontualmente:
MAX_TOOL_ITERATIONS=20 tsx scripts/run-agent.ts outreach-agent "..."
```

```
# Desabilitar cheap mode para tarefa complexa:
DEV_MODE=false tsx scripts/run-agent.ts outreach-agent "..."
```

```
# Ser mais específico no prompt:
tsx scripts/run-agent.ts outreach-agent "Processe APENAS a empresa AWS. Gere um pacote de outreach para Rachel Wilson, Head of Marketing LATAM, score 88."
```

Problema: Unknown tool: `save_lead`

Causa: Tool não registrada no agente.

Solução: Verificar o método `static async create()` do agente e confirmar que `agent.registerTool(...)` é chamado para a tool em questão.

13.6 Problemas de Performance

Problema: Agente lento (30+ segundos por resposta)

Causas e soluções:

Causa	Solução
Modelo muito poderoso para tarefa simples	Definir <code>DEV_MODE=true</code> OU <code>--model claude-haiku-4-5-20251001</code>
Context muito grande (80k+ tokens)	Habilitar <code>contextCompression</code> (automático quando <code>> limit</code>)
Cache miss em system prompt longo	Aguardar — após 1º use, <code>cacheRead</code> será usado (10× mais rápido)
Redis indisponível (embedding cache miss)	Subir infraestrutura: <code>npm run infra:up</code>
Muitas tool calls em sequência	Reduzir escopo da tarefa no prompt

14. Deployment em Outras Empresas

14.1 Adaptação do Sistema

A plataforma AI Cognitive Runtime é projetada para ser adaptável. Para deployar para outra empresa:

Passo 1: Fork e configuração base

```
git clone <repo>
cd ai-cognitive-runtime

# Renomear para a empresa alvo
# Editar package.json name

cp .env.example .env
# Preencher com keys da empresa
```

Passo 2: Substituir identidade VRASHOWS

Todos os textos de posicionamento estão em:

- `prompts/agents/outreach-agent.md` — identidade da empresa
- `prompts/agents/email-sender-agent.md` — remetente e positioning
- `prompts/agents/lead-enrichment-agent.md` — contexto de quem contratar
- `prompts/agents/futurecom-researcher.md` — segmentos alvo
- `assets/templates/*.md` — templates de email

Substituições típicas:

- Nome da empresa
- Tagline
- Segmentos-alvo
- Eventos-alvo
- Caso de referência (equivalente ao "Brasil TecPar / ABRINT 2026")
- Email e domínio corporativo

Passo 3: Configurar domínio de email

```
# 1. Criar conta no Resend (resend.com)
# 2. Adicionar domínio da empresa
# 3. Configurar 3 DNS records (TXT, MX, TXT)
# 4. Aguardar propagação (24-48h)
# 5. Verificar via API ou dashboard

# .env:
RESEND_FROM_EMAIL=contato@empresa.com.br
RESEND_FROM_NAME=Nome da Empresa
```

Passo 4: Atualizar domain registry no EmailPatternResolver

Em `agents/lead-enrichment-agent/email-resolver.ts`, adicionar entradas para empresas relevantes ao setor da empresa:

```
const KNOWN_DOMAINS: Record<string, DomainEntry> = {
  // Adicionar empresas do setor:
  "nome-empresa-alvo": {
    domain: "empresa-alvo.com.br",
    primaryPattern: "firstname.lastname",
    patternConfidence: "high",
  },
  // ...
};
```

Passo 5: Adaptar leads e templates

```
# Criar novo arquivo de leads
data/leads/<empresa>-leads.json

# Adaptar templates para o segmento
assets/templates/cold-outreach.md
assets/templates/executive-intro.md
```

Passo 6: Configurar media kit

```
# Copiar material institucional da empresa
assets/pdfs/<empresa>_media_kit.pdf
```

```
# Atualizar .env
MEDIA_KIT_PDF=./assets/pdfs/<empresa>_media_kit.pdf
```

14.2 Configuração de Produção

Para ambiente de produção (não dev):

```
# Produção – sem cheap mode
DEV_MODE=false
CHEAP_MODE=false

# Modelos mais capazes
DEFAULT_MODEL=claude-sonnet-4-6
FAST_MODEL=claude-haiku-4-5-20251001
POWERFUL_MODEL=claude-sonnet-4-6

# Memória habilitada
ENABLE_MEMORY=true
DATABASE_URL=postgresql://...
OPENAI_API_KEY=sk-proj-...

# Limites de produção
MAX_TOOL_ITERATIONS=10
MAX_OUTPUT_TOKENS=8192

# Log mínimo
LOG_LEVEL=info
```

14.3 Checklist de Go-Live

- ☐ .env configurado com keys de produção
- ☐ Domínio de email verificado no Resend
- ☐ DNS records propagados (TXT DKIM, MX, TXT SPF)
- ☐ PDF media kit em assets/pdfs/
- ☐ Prompts de agentes revisados com identidade da empresa
- ☐ Templates revisados com posicionamento correto
- ☐ Domain registry atualizado com empresas-alvo do setor
- ☐ npm run typecheck → zero erros
- ☐ Teste de email para email interno confirmado
- ☐ Infra (Redis + Postgres) up e healthy
- ☐ Primeiro lead enrichment executado e validado
- ☐ Custo de primeiro run monitorado
- ☐ .gitignore verificado (.env, assets/pdfs, logs)
- ☐ Variáveis sensíveis nunca no código ou commits

14.4 Escalabilidade

A plataforma suporta escalonamento horizontal:

Redis: Usar Redis Cloud ou Upstash para Redis managed (trocar `REDIS_URL`)

PostgreSQL: Usar Supabase, Neon, ou AWS RDS com pgvector. Ambos suportam `vector` extension.

Agentes: Cada agent é stateless entre runs. Múltiplas instâncias podem processar em paralelo via `workflows/` .

Email: O Resend suporta múltiplos domínios e regiões. Para volume alto, considerar:

- Resend Broadcasts para campanhas
- Múltiplas API keys rotacionadas

15. Roadmap Futuro

15.1 Curto Prazo (1-3 meses)

15.1.1 DNS Verification — `vrashows.com.br`

Status: Pendente

Blocker: 3 DNS records precisam ser adicionados ao registrar

Após verificação:

- Emails enviados do domínio `vrashows.com.br` com SPF, DKIM e MX corretos
- Deliverability significativamente melhor (menos spam)
- Rastreamento de abertura via Resend dashboard

15.1.2 Outreach Pipeline Automatizado

Objetivo: Pipeline completo: pesquisa → enriquecimento → outreach → envio

```
FuturecomResearcherAgent
  ↓ leads[]
LeadEnrichmentAgent
  ↓ contacts[]
OutreachAgent
  ↓ packages[]
EmailSenderAgent
  ↓ records[]
Dashboard / logs
```

Implementação: `workflows/outreach-pipeline.ts`

```
export async function runOutreachPipeline(event: string, companies: string[]) {
  const researcher = await FuturecomResearcherAgent.create();
  const enricher = await LeadEnrichmentAgent.create();
  const outreach = await OutreachAgent.create();
  const sender = await EmailSenderAgent.create();

  const leads = await researcher.research(companies, event);
  const contacts = await enricher.enrich(leads);
  const packages = await outreach.generate(contacts.map(toLeadProfile));
  const records = await sender.send(packages);

  return { leads, contacts, packages, records };
}
```

15.1.3 Resend Webhook para Rastreamento

Objetivo: Capturar eventos de abertura, clique e resposta via webhook Resend

Eventos a rastrear:

- email.opened → atualizar outreachStatus: "opened"
- email.clicked → sinal de interesse alto
- email.bounced → marcar email como inválido
- email.complained → descadastrar imediatamente

Implementação: Endpoint HTTP (Express) + atualização do aws-leads.json e Redis

15.1.4 Dashboard de Custos e Métricas

Objetivo: Interface web leve para monitorar em tempo real:

- Custo por agente por dia
- Emails enviados / abertos / respondidos
- Leads em cada estágio do pipeline
- Taxa de conversão de contato para reunião

15.2 Médio Prazo (3-6 meses)

15.2.1 Multi-Event Support

Generalizar o FuturecomResearcherAgent para qualquer evento:

```
// Atual:
tsx scripts/run-agent.ts futurecom-researcher "Pesquise leads para Futurecom 2026"

// Futuro:
tsx scripts/run-agent.ts event-researcher \
  --event "ABRINT 2027" \
  --date "2027-06-15" \
  --city "Florianópolis" \
  "Pesquise e qualifique leads"
```

15.2.2 LinkedIn Outreach Automation

Adicionar linkedinMessage como canal real, não apenas texto gerado:

- Tool send_linkedin_message via API do LinkedIn ou automação
- Rate limiting: 50 mensagens/semana (limite do LinkedIn)
- Personalização por conexão em comum

15.2.3 CRM Integration

Exportar leads e registros de outreach para CRMs:

CRM	Integração
HubSpot	REST API — criar contact, deal, activity
Pipedrive	REST API — criar person, deal, note
Notion	API — atualizar database de leads

Google Sheets

Sheets API — append rows

15.2.4 Email Pattern Resolver v2

Melhorias no `EmailPatternResolver` :

- Validação MX record (DNS lookup sem scraping)
- Integração com Hunter.io ou Apollo.io (opcional, via API key)
- Mais 60+ empresas no registry (expansão para 100+)
- Suporte a padrões regionais (LATAM: Chile, México, Colombia)
- Score de confiança mais granular (0-100 em vez de 3 níveis)

15.3 Longo Prazo (6-12 meses)

15.3.1 Autonomous Campaign Manager

Agente coordinator que executa campanhas autônomas:

1. Monitora calendário de eventos
2. Pesquisa e qualifica leads automaticamente
3. Gera e aprova conteúdo (com human-in-the-loop para revisão)
4. Despacha emails no timing ideal
5. Faz follow-up baseado em abertura/resposta
6. Reporta pipeline semanal

15.3.2 LLM-Powered Lead Scoring

Substituir o scoring heurístico por um modelo fine-tuned:

- Dataset: leads históricos + conversão real
- Features: cargo, empresa, evento, tamanho, setor, budget
- Output: score 0-100 + reason + propensão a responder

15.3.3 Multi-Tenant Architecture

Adaptar para múltiplos clientes (SaaS):

- Tenant isolation por `tenantId` em todas as tabelas
- System prompts por tenant (identidade de empresa)
- Billing por token/agent/month
- Dashboard multi-tenant

15.3.4 RAG sobre Pipeline de Outreach

Usar o vault e memória para:

- Recuperar o que funcionou em campanhas anteriores (subject lines, variantes de CTA)
- Adaptar posicionamento baseado em respostas recebidas
- Identificar padrões de conversão por segmento

15.3.5 Eval Framework Completo

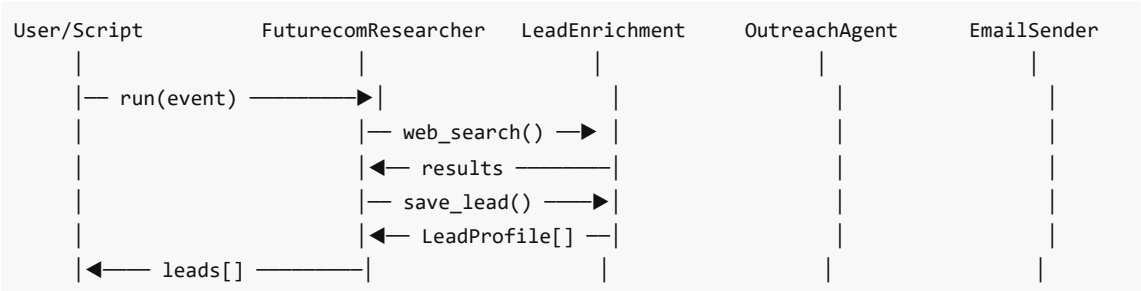
Expandir `evals/` para avaliar automaticamente qualidade de:

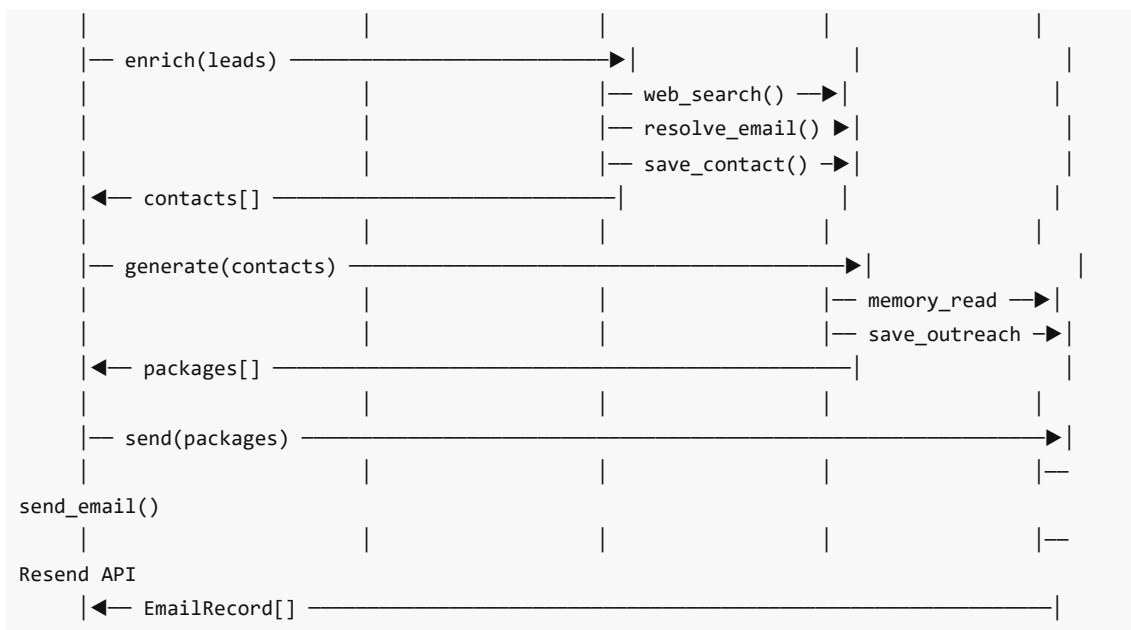
- Emails gerados (clareza, posicionamento, CTA)
 - Leads identificados (relevância, prioridade)
 - Emails inferidos (acurácia de email pattern)
 - Custo vs. qualidade (quality/cost ratio por agente)
-

Apêndice A — Glossário

Termo	Definição
Agentic loop	Loop while(iterations < max) onde o agente chama a API, processa tool calls, e repete até end_turn
Cache control	Diretiva cache_control: { type: "ephemeral" } no system prompt para caching de prompt pela Anthropic
Cheap mode	DEV_MODE=true CHEAP_MODE=true → Haiku default, 2048 tokens max, 5 iterations max
Cold outreach	Primeiro contato com um prospect (sequenceNumber=1)
Cosine similarity	Métrica de similaridade vetorial usada pelo pgvector (0=ortogonal, 1=idêntico)
EmailRecord	Struct de resultado de envio: company, status, resendId, sentAt, error
guessedEmails[]	Array de emails inferidos ordenados por confiança (decrescente)
IVFFlat	Tipo de índice pgvector (Inverted File with Flat Quantization) — approximated search
Lean mode	ENABLE_MEMORY=false — todos os componentes de memória são no-ops, sem crash
LeadProfile	Struct de lead pesquisado pelo FuturecomResearcherAgent
Memory injection	Adição automática de memórias relevantes ao system prompt antes de cada call
Model router	Componente que decide qual modelo usar baseado em heurísticas + complexidade
OutreachPackage	Struct com email + LinkedIn message gerados pelo OutreachAgent
pgvector	Extensão PostgreSQL para armazenamento e busca de vetores de embeddings
Prompt caching	Mecanismo da API Anthropic que reutiliza tokens do system prompt (10× mais barato)
Rate limiting	Delay de 1200ms entre envios de email para respeitar limites do Resend
Resend	Plataforma de email transacional usada para envio de outreach
ToolHandler	Interface { name, schema, execute() } que define uma tool disponível ao agente
tsx	TypeScript executor direto (sem transpilação) — npx tsx file.ts

Apêndice B — Diagrama de Sequência: Pipeline Completo





Apêndice C — Configuração de Desenvolvimento Rápido

Para iniciar o desenvolvimento em um novo ambiente em menos de 5 minutos:

```
# Clonar e instalar
git clone <repo>
cd ai-cognitive-runtime
npm install

# Configurar .env mínimo (apenas Anthropic API Key)
cat > .env << 'EOF'
ANTHROPIC_API_KEY=sk-ant-api03-...
ENABLE_MEMORY=false
DEV_MODE=true
CHEAP_MODE=true
MAX_TOOL_ITERATIONS=5
MAX_OUTPUT_TOKENS=2048
LOG_LEVEL=info
EOF

# Verificar tipos
npm run typecheck

# Teste imediato (sem infraestrutura)
tsx scripts/run-agent.ts researcher "Hello, are you working?"

# Output esperado:
# --- OUTPUT ---
# Yes, I'm operational! I'm an AI researcher agent...
# [cost] $0.000123 saved=$0.000000
# [tokens] in:847 out:47 cache_read:0 cache_write:0
```

Documento gerado em 2026-05-19. Manter sincronizado com cada major release da plataforma.

Para atualizar: `docs/AI_RUNTIME_ENTERPRISE_MANUAL.md` — versão markdown canônica.